2ND LABORATORY REPORT

From NFA to minimized DFA

Author
Flores Cóngora, Paolo Luis

Mentor
Adrián Martínez Manzo

National Autonomous University of Mexico, Computer Science: Compilers

Contents

1 Introduction . . . 3
1.1 Problem Statement . . . 3
2 Implementation . . . 4
2.1 Data Structures . . . 4
2.2 NFA to DFA Conversion . . . 4
2.2.1 Subset Construction Algorithm . . . 4
2.3 DFA Minimization . . . 5
2.3.1 Initial Partition . . . 5
2.3.2 Partition Refinement . . . 6
2.3.3 Building the Minimized DFA . . . 6
2.4 Visualization . . . 8

1. Introduction

In the last assignment, we learnt how to build an NFA and evaluate it for user input. The goal now is to create a program that converts that NFA to a DFA, and finally use a minimization algorithm to further reduce the automata's size, making the evaluation step faster and smaller.

1.1 Problem Statement

Given an NFA constructed from a regular expression, the program must:

1. Convert the NFA to an equivalent DFA using subset construction algorithm
2. Minimize the DFA using the partition refinement algorithm
3. Construct the minimized DFA from the final partition
4. Visualize all automata using `graphviz`

2. Implementation

2.1 Data Structures

The implementation now uses Python instead of C. The equivalent data structures from last report are the following:

```
EPSILON = 'ε'

class State:
    def __init__(self, name: str):
        self.name = name
        self.transitions = []  # list of (to_state, symbol)
        self.is_accepting = False

    def add_transition(self, to_state, symbol):
        self.transitions.append((to_state, symbol))
```

The automata is represented as a collection of states with a designated start state:

```
class NFA:
    def __init__(self):
        self.start_state = None
        self.states = set()

    def get_alphabet(self):
        return sorted({
            sym for s in self.states
            for _, sym in s.transitions
            if sym != EPSILON
        })
```

This structure is reused for DFAs.

2.2 NFA to DFA Conversion

The subset construction algorithm converts an NFA into an equivalent DFA. Each DFA state represents a set of NFA states that can be simultaneously active.

2.2.1 Subset Construction Algorithm

The algorithm maintains a worklist of unmarked DFA states. Each DFA state is represented as a `frozenset` of NFA states for hashing purposes.

The algorithm starts by computing the ε-closure of the NFA's start state as the initial DFA state. Then, for each unmarked DFA state T:

- For each symbol a in the alphabet, compute $U = \varepsilon\text{-closure}(\text{move}(T, a))$
- If U is non-empty and not already a DFA state, create it and add to the worklist
- Add a transition from T to U on symbol a

The process continues until no unmarked states remain.

```python
def nfa_to_dfa(nfa):
    alphabet = nfa.get_alphabet()

    # Initial DFA state = ε-closure of NFA start
    start_set = frozenset(epsilon_closure_single(nfa.start_state))
    dfa_states = {start_set: State("D0")}
    dfa_states[start_set].is_accepting = any(s.is_accepting for s in start_set)

    unmarked = [start_set]
    state_count = 1

    while unmarked:
        T = unmarked.pop()
        for symbol in alphabet:
            U = frozenset(epsilon_closure_set(move(T, symbol)))

            if not U:
                continue

            if U not in dfa_states:
                new_state = State(f"D{state_count}")
                new_state.is_accepting = any(s.is_accepting for s in U)
                dfa_states[U] = new_state
                unmarked.append(U)
                state_count += 1

            dfa_states[T].add_transition(dfa_states[U], symbol)

    # Build result automaton
    dfa = NFA()
    dfa.start_state = dfa_states[start_set]
    dfa.states = set(dfa_states.values())
    return dfa
```

A DFA state is marked as accepting if any of its constituent NFA states is accepting.

2.3 DFA Minimization

The minimization algorithm uses partition refinement to identify equivalent states. Two states are equivalent if they have the same accepting status and transition to equivalent states on all input symbols.

Reference video: https://youtu.be/0XaGAkY09Wc?si=MFUjs0Tl4gPnamUk

2.3.1 Initial Partition

The first step separates accepting and non-accepting states into two groups:

```python
def get_initial_partition(dfa_states):
    accepting = {s for s in dfa_states if s.is_accepting}
    non_accepting = {s for s in dfa_states if not s.is_accepting}
    return [accepting, non_accepting]
```

2.3.2 Partition Refinement

The `refine` function splits groups when states don't have the same equivalence level. Each state receives a signature based on which partition group its transitions lead to.

- States with identical signatures remain together
- States with different signatures are separated

And this function is meant to be called until the changed result is `False`.

```
def refine(partition, alphabet):
    state_to_group = {s: idx for idx, g in enumerate(partition) for s in g}
    new_partition = []
    changed = False

    for group in partition:
        subgroups = {}

        for state in group:
            signature = []
            for symbol in alphabet:
                target = None
                for to, sym in state.transitions:
                    if sym == symbol:
                        target = to
                        break
                signature.append(
                    state_to_group[target] if target else None
                )

            signature = tuple(signature)
            subgroups.setdefault(signature, set()).add(state)

        if len(subgroups) > 1:
            changed = True
        new_partition.extend(subgroups.values())

    return new_partition, changed
```

2.3.3 Building the Minimized DFA

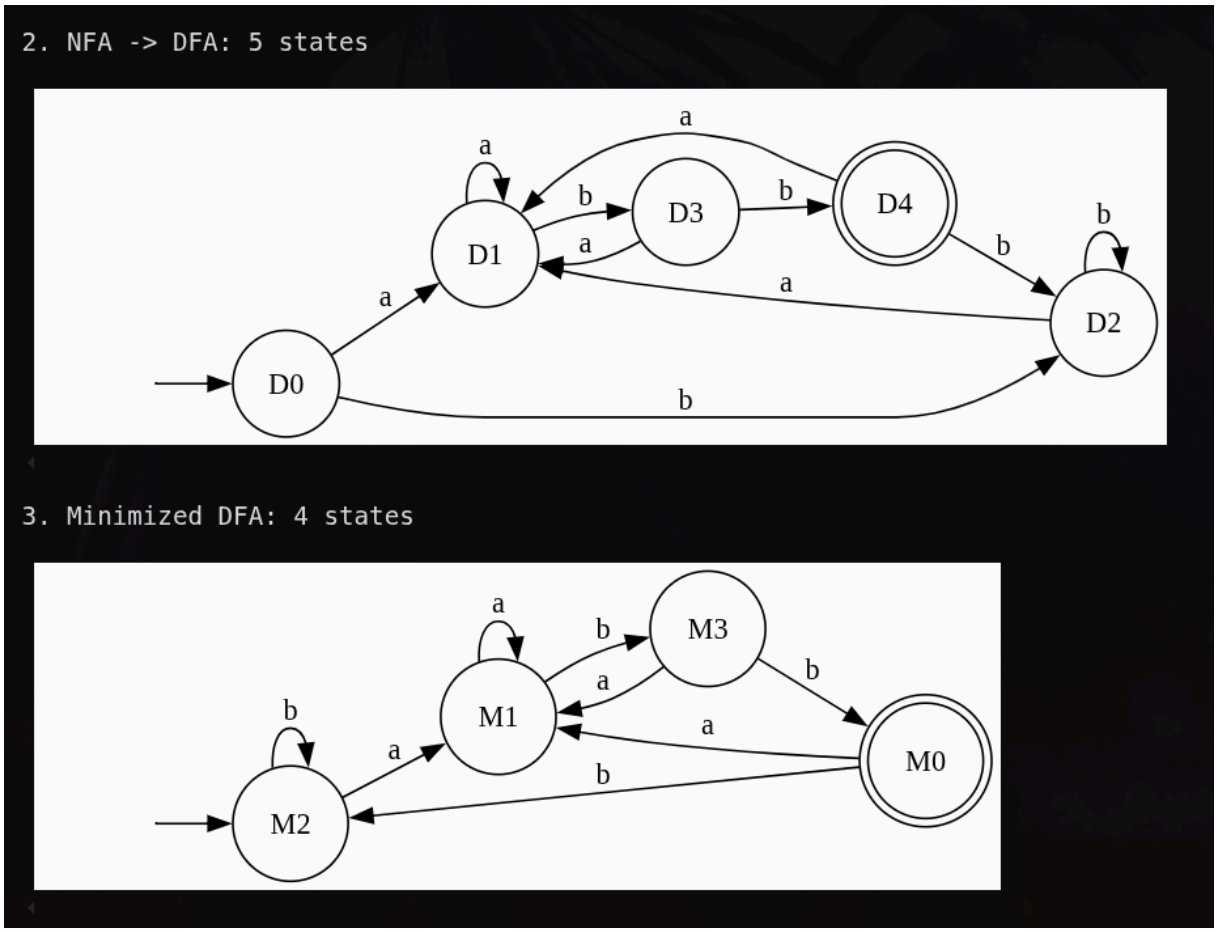
After refinement converges, each partition group becomes a single state in the minimized DFA.

Composing the minimized DFA is simple:

- Each partition group becomes a single state in the minimized DFA.
- The state is accepting if any of its constituent states is accepting.
- The transitions are the same as the original DFA.
- The start state is the one that corresponds to the partition group of the original start state.

```
def minimize_dfa(dfa):
    alphabet = dfa.get_alphabet()
    partition = [g for g in get_initial_partition(dfa.states) if g]

    changed = True
    while changed:
```

```
        partition, changed = refine(partition, alphabet)

    # Map original states to group index
    state_to_group = {s: i for i, g in enumerate(partition) for s in g}

    # Create one state per group
    new_states = [State(f"M{i}") for i in range(len(partition))]

    for idx, group in enumerate(partition):
        new_states[idx].is_accepting = any(s.is_accepting for s in group)

        # Use any representative to get transitions
        representative = next(iter(group))
        for to_state, symbol in representative.transitions:
            target_idx = state_to_group[to_state]
            new_states[idx].add_transition(new_states[target_idx], symbol)

    minimized = NFA()
    minimized.start_state = new_states[state_to_group[dfa.start_state]]
    minimized.states = set(new_states)
    return minimized
```



The key insight is that all states within a group have identical transition behavior (same level of equivalence), so any representative determines the group's transitions.

2.4 Visualization

The `graphviz` library renders automata as SVG diagrams:

```
def draw(automaton):
    dot = graphviz.Digraph(format='svg', graph_attr={'rankdir': 'LR'})

    for s in automaton.states:
        shape = 'doublecircle' if s.is_accepting else 'circle'
        dot.node(s.name, s.name, shape=shape)

    # Invisible node for start arrow
    dot.node('', '', shape='none')
    dot.edge('', automaton.start_state.name)

    for s in automaton.states:
        for to, sym in s.transitions:
            dot.edge(s.name, to.name, label=sym)

    return dot
```